

pohsun__40183452

October 22, 2025

#EECS 442 PS6: Scene Recognition

Please provide the following information (e.g. Aparajito Saha, rajsaha):

Po-Hsun Chang, pohsun

Important: after you download the .ipynb file, please name it as ____<your_uniqname>__<your_umid>.ipynb__ before you submit it to canvas. Example: rajsaha__01101100.ipynb.

1 Starting

Run the following code to import the modules you'll need. After your finish the assignment, remember to run all cells and save the note book to your local machine as a .ipynb file for Canvas submission.

```
[ ]: import pickle
import numpy as np
import matplotlib.pyplot as plt
import os
import copy
from tqdm import tqdm

import torch
import torchvision
from torchvision import datasets, models, transforms
import torch.nn as nn
import torch.optim as optim

print("PyTorch Version: ",torch.__version__)
print("Torchvision Version: ",torchvision.__version__)
# Detect if we have a GPU available
device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
if torch.cuda.is_available():
    print("Using the GPU!")
else:
    print("WARNING: Could not find GPU! Using CPU only. If you want to enable_
↳GPU, please to go Edit > Notebook Settings > Hardware Accelerator and select_
↳GPU.")
```

```
data_dir = "./data_miniplaces_modified"
```

PyTorch Version: 2.8.0+cu126
Torchvision Version: 0.23.0+cu126
Using the GPU!

2 Problem 6.1 Scene Recognition with VGG

You will build and train a convolutional neural network for scene recognition, i.e., classifying images into different scenes. You will need to: 1. Construct dataloaders for train/val/test datasets 2. Build MiniVGG and MiniVGG-BN (MiniVGG with batch-normalization layers) 3. Train MiniVGG and MiniVGG-BN, compare their training progresses and their final top-1 and top-5 accuracies. 4. Increase the size of the network by adding more layers and check whether top-1 and top-5 accuracies will improve.

2.1 Step 0: Downloading the dataset.

```
[ ]: # Download the miniplaces dataset
# Note: Restarting the runtime won't remove the downloaded dataset. You only
      ↳ need to re-download the zip file if you lose connection to colab.
!wget http://www.eecs.umich.edu/courses/eecs504/data_miniplaces_modified.zip
```

```
--2025-10-17 16:07:27--
http://www.eecs.umich.edu/courses/eecs504/data_miniplaces_modified.zip
Resolving www.eecs.umich.edu (www.eecs.umich.edu)... 141.212.113.116
Connecting to www.eecs.umich.edu (www.eecs.umich.edu)|141.212.113.116|:80...
connected.
HTTP request sent, awaiting response... 200 OK
Length: 534628730 (510M) [application/zip]
Saving to: 'data_miniplaces_modified.zip'

data_miniplaces_mod 100%[=====>] 509.86M  37.7MB/s   in 34s

2025-10-17 16:08:01 (14.9 MB/s) - 'data_miniplaces_modified.zip' saved
[534628730/534628730]
```

```
[ ]: # Unzip the download dataset .zip file to your local colab dir
# Warning: this unzipping process may take a while. Please be patient.
!unzip -q data_miniplaces_modified.zip
```

```
[ ]: from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

2.2 6.1 (a): Build dataloaders for train, val, and test

```
[ ]: def get_dataloaders(input_size, batch_size, shuffle = True):
    """
    Build dataloaders with transformations.

    Args:
        input_size: int, the size of the tranformed images
        batch_size: int, minibatch size for dataloading

    Returns:
        dataloader_dict: dict, dict with "train", "val", "test" keys, each is_
        ↪ mapped to a pytorch dataloader.
    """

    mean = [0.485, 0.456, 0.406]
    std = [0.229, 0.224, 0.225]

    #####
    # TODO: Step 1: Build transformations for the dataset. #
    # You need to construct a data transformation that does three #
    # preprocessing steps in order: #
    # I. Resize the image to input_size using transforms.Resize #
    # II. Convert the image to PyTorch tensor using transforms.ToTensor #
    # III. Normalize the images with the provided mean and std parameters #
    # using transforms.Normalize. These parameters are accumulated from a #
    # large number of training samples. #
    # You can use transforms.Compose to combine the above three #
    # transformations. Store the combined transforms in the variable #
    # 'composed_transform'. #
    #####

    composed_transform = transforms.Compose([
        transforms.Resize((input_size, input_size)),
        transforms.ToTensor(),
        transforms.Normalize(mean=mean, std=std)
    ])

    #####
    #                                     END OF YOUR CODE #
    #####

    # We write the remaining part of the dataloader for you.
    # You are encouraged to go through this.

    #####
    # Step 2: Build dataloaders. #
```

```

# I. We use torch.datasets.ImageFolder with the provided data_dir and the #
# data transformations you created in step 1 to construct pytorch datasets #
# for training, validation, and testing.                                #
# II. Then we use torch.utils.data.DataLoader to build dataloaders with #
# the constructed pytorch datasets. You need to enable shuffling for    #
# the training set. Set num_workers=2 to speed up dataloading.          #
# III. Finally, we put the dataloaders into a dictionary.                #
#####

# Create train/val/test datasets
data_transforms = {
    'train': composed_transform,
    'val': composed_transform,
    'test': composed_transform
}

image_datasets = {x: datasets.ImageFolder(os.path.join(data_dir, x),
data_transforms[x]) for x in data_transforms.keys()}

# Create training train/val/test dataloaders
# Never shuffle the val and test datasets
dataloaders_dict = {x: torch.utils.data.DataLoader(image_datasets[x],
batch_size=batch_size, shuffle=False if x != 'train' else shuffle,
num_workers=2) for x in data_transforms.keys()}

return dataloaders_dict

```

```

[ ]: batch_size = 16
input_size = 128
dataloaders_dict = get_dataloaders(input_size, batch_size)

# Confirm your train/val/test sets contain 90,000/10,000/10,000 samples
print('# of training samples {}'.format(len(dataloaders_dict['train'].dataset)))
print('# of validation samples {}'.format(len(dataloaders_dict['val'].dataset)))
print('# of test samples {}'.format(len(dataloaders_dict['test'].dataset)))

```

```

# of training samples 90000
# of validation samples 10000
# of test samples 10000

```

```

[ ]: # Visualize the data within the dataset
import json
with open('./data_miniplaces_modified/category_names.json', 'r') as f:
    class_names = json.load(f)['i2c']
class_names = {i:name for i, name in enumerate(class_names)}

def imshow(inp, title=None, ax=None, figsize=(10, 10)):
    """Imshow for Tensor."""

```

```

inp = inp.numpy().transpose((1, 2, 0))
mean = np.array([0.485, 0.456, 0.406])
std = np.array([0.229, 0.224, 0.225])
inp = std * inp + mean
inp = np.clip(inp, 0, 1)
if ax is None:
    fig, ax = plt.subplots(1, figsize=figsize)
ax.imshow(inp)
ax.set_xticks([])
ax.set_yticks([])
if title is not None:
    ax.set_title(title)

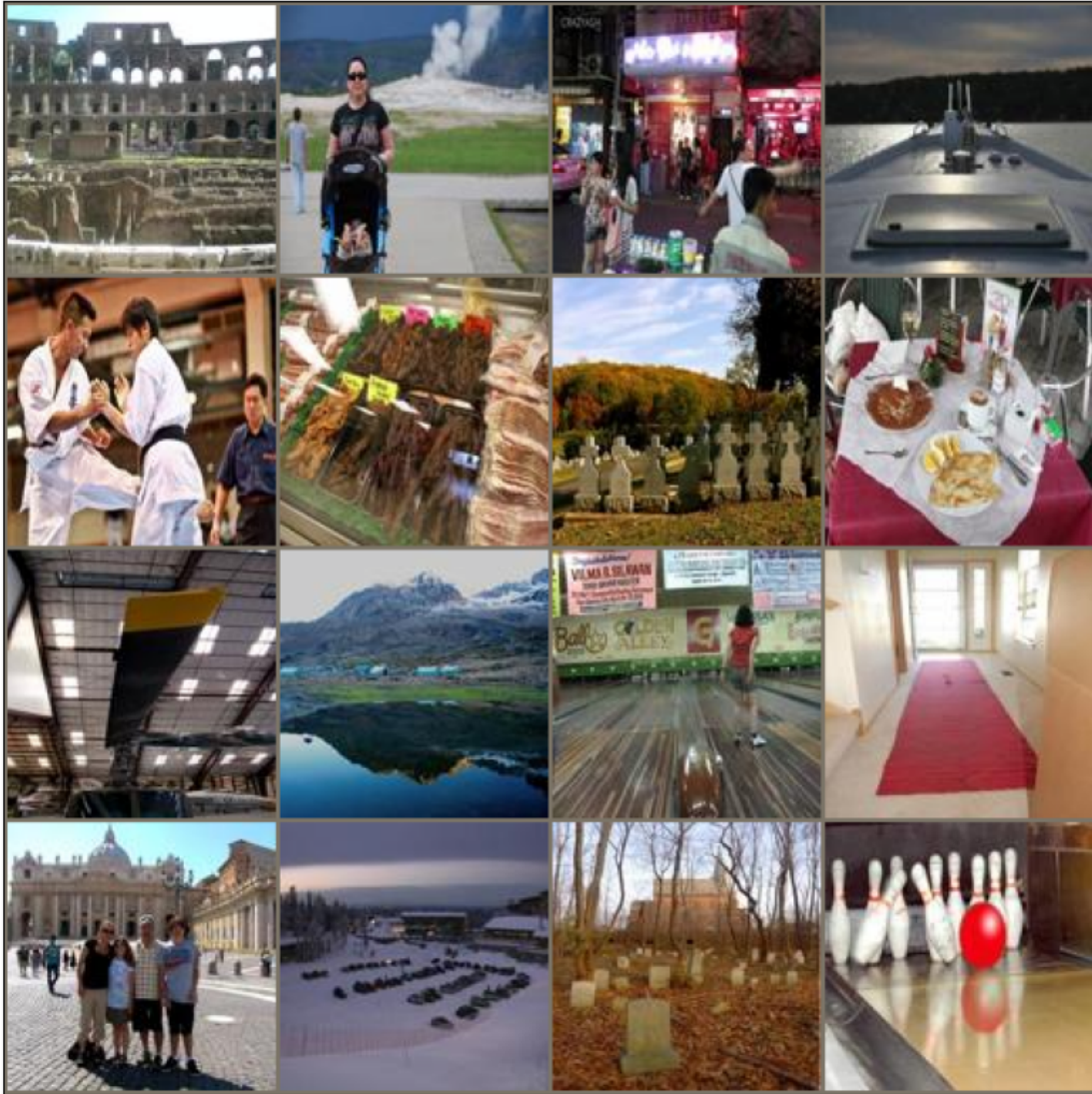
# Get a batch of training data
inputs, classes = next(iter(dataloaders_dict['train']))

# Make a grid from batch
out = torchvision.utils.make_grid(inputs, nrow=4)

fig, ax = plt.subplots(1, figsize=(10, 10))
title = [class_names[x.item()] if (i+1) % 4 != 0 else class_names[x.
    ↪item()]+\n' for i, x in enumerate(classes)]
imshow(out, title=' | '.join(title), ax=ax)

```

amphitheater | hot_spring | bar | boat_deck
 | martial_arts_gym | butchers_shop | cemetery | restaurant
 | museum | campsite | bowling_alley | corridor
 | monastery | ski_slope | cemetery | bowling_alley



2.3 6.1 (b): Build MiniVGG and MiniVGG-BN

Please follow the instructions to build the two neural networks with architectures shown below.

MiniVGG architecture

MiniVGG-BN architecture

```
[ ]: # Helper function for counting number of trainable parameters.
def count_params(model):
    """
    Counts the number of trainable parameters in PyTorch.

    Args:
        model: PyTorch model.

    Returns:
        num_params: int, number of trainable parameters.
    """

    num_params = sum([item.numel() for item in model.parameters() if item.
↪requires_grad])

    return num_params
```

```
[ ]: # Network configurations for all layers before the final fully-connected layers.
# "M" corresponds to maxpooling layer, integers correspond to number of output
# channels of a convolutional layer.
cfgs = {
    'MiniVGG': [64, 'M', 128, 'M', 128, 128, 'M'],
    'MiniVGG-BN': [64, 'M', 128, 'M', 128, 128, 'M']
}

def make_layers(cfg, batch_norm=False):
    """
    Return a nn.Sequential object containing all layers to get the features
    using the CNN. (That is, before the Average pooling layer in the two
    pictures above).

    Args:
        cfg: list
        batch_norm: bool, default: False. If set to True, a BatchNorm layer
            should be added after each convolutional layer.

    Return:
        features: torch.nn.Sequential. Containers for all feature extraction
            layers. For use of torch.nn.Sequential, please refer to
            PyTorch documentation.
    """
    #####
    # TODO: Construct the neural net architecture from cfg. You should use #
    # nn.Sequential(). #
    #####
```

```

features = nn.Sequential()
in_channels = 3
for layer in cfg:
    if layer == 'M':
        features.append(nn.MaxPool2d(kernel_size=2, stride=2))
    else:
        features.append(nn.Conv2d(in_channels, out_channels=layer,
kernel_size=3, padding=1))
        if batch_norm:
            features.append(nn.BatchNorm2d(num_features=layer))
        features.append(nn.ReLU(inplace=False))
        in_channels = layer

#####
#                                     END OF YOUR CODE                                     #
#####

return features

class VGG(nn.Module):

    def __init__(self, features, num_classes=100, init_weights=True):
        super(VGG, self).__init__()

        self.features = features
        self.avgpool = nn.AdaptiveAvgPool2d((5, 5))

        #####
        # TODO: Construct the final FC layers using nn.Sequential.                        #
        # Note: The average pooling layer has been defined by us above.                  #
        #####

        self.classifier = nn.Sequential(
            nn.Linear(in_features=3200, out_features=512),
            nn.ReLU(inplace=False),
            nn.Dropout(p=0.5, inplace=False),
            nn.Linear(in_features=512, out_features=256),
            nn.ReLU(inplace=False),
            nn.Dropout(p=0.5, inplace=False),
            nn.Linear(in_features=256, out_features=100)
        )

        #####
        #                                     END OF YOUR CODE                                     #
        #####

```



```

        if init_weights:
            self._initialize_weights()

    def forward(self, x):
        x = self.features(x)
        x = self.avgpool(x)
        x = torch.flatten(x, 1)
        x = self.classifier(x)
        return x

    def _initialize_weights(self):
        for m in self.modules():
            if isinstance(m, nn.Conv2d):
                nn.init.kaiming_normal_(m.weight, mode='fan_out',
nonlinearity='relu')
                if m.bias is not None:
                    nn.init.constant_(m.bias, 0)
            elif isinstance(m, nn.BatchNorm2d):
                nn.init.constant_(m.weight, 1)
                nn.init.constant_(m.bias, 0)
            elif isinstance(m, nn.Linear):
                nn.init.normal_(m.weight, 0, 0.01)
                nn.init.constant_(m.bias, 0)

```

```

[ ]: features = make_layers(cfgs['MiniVGG'], batch_norm=False)
    vgg = VGG(features)

features = make_layers(cfgs['MiniVGG-BN'], batch_norm=True)
vgg_bn = VGG(features)

# Print the network architecture. Please compare the printed architecture with
# the one given in the instructions above.
# Make sure your network has the same architecture as the one we give above.
print(vgg)
print('Number of trainable parameters {}'.format(count_params(vgg)))

print(vgg_bn)
print('Number of trainable parameters {}'.format(count_params(vgg_bn)))

```

```

VGG(
  (features): Sequential(
    (0): Conv2d(3, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (1): ReLU()
    (2): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1,
ceiling_mode=False)
    (3): Conv2d(64, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (4): ReLU()

```

```

        (5): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1,
ceil_mode=False)
        (6): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
        (7): ReLU()
        (8): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
        (9): ReLU()
        (10): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1,
ceil_mode=False)
    )
    (avgpool): AdaptiveAvgPool2d(output_size=(5, 5))
    (classifier): Sequential(
        (0): Linear(in_features=3200, out_features=512, bias=True)
        (1): ReLU()
        (2): Dropout(p=0.5, inplace=False)
        (3): Linear(in_features=512, out_features=256, bias=True)
        (4): ReLU()
        (5): Dropout(p=0.5, inplace=False)
        (6): Linear(in_features=256, out_features=100, bias=True)
    )
)
Number of trainable parameters 2166756
VGG(
    (features): Sequential(
        (0): Conv2d(3, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
        (1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
        (2): ReLU()
        (3): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1,
ceil_mode=False)
        (4): Conv2d(64, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
        (5): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
        (6): ReLU()
        (7): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1,
ceil_mode=False)
        (8): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
        (9): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
        (10): ReLU()
        (11): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
        (12): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
        (13): ReLU()
        (14): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1,
ceil_mode=False)
    )
    (avgpool): AdaptiveAvgPool2d(output_size=(5, 5))
    (classifier): Sequential(

```

```

(0): Linear(in_features=3200, out_features=512, bias=True)
(1): ReLU()
(2): Dropout(p=0.5, inplace=False)
(3): Linear(in_features=512, out_features=256, bias=True)
(4): ReLU()
(5): Dropout(p=0.5, inplace=False)
(6): Linear(in_features=256, out_features=100, bias=True)
)
)
Number of trainable parameters 2167652

```

2.4 6.1 (c): Build training/validation loops

You will write a function for training and validating the network.

```

[ ]: def make_optimizer(model):
    """
    Args:
        model: NN to train

    Returns:
        optimizer: pytorch optimizer for updating the given model parameters.
    """
    #####
    # TODO: Create a SGD optimizer with learning rate=1e-2 and momentum=0.9. #
    # HINT: Check out optim.SGD() and initialize it with the appropriate #
    # parameters. We have imported torch.optim as optim for you above. #
    #####
    optimizer = optim.SGD(model.parameters(), lr=1e-2, momentum=0.9)
    #####
    #                                     END OF YOUR CODE                                     #
    #####
    return optimizer

def get_loss():
    """
    Returns:
        criterion: pytorch loss.
    """
    #####
    # TODO: Create an instance of the cross entropy loss. This code #
    # should be a one-liner. #
    #####
    criterion = nn.CrossEntropyLoss()
    #####
    #                                     END OF YOUR CODE                                     #
    #####

```

```
return criterion
```

```
[ ]: def train_model(model, dataloaders, criterion, optimizer, save_dir = None,
↳ num_epochs=25, model_name='MiniVGG'):
    """
    Args:
        model: The NN to train
        dataloaders: A dictionary containing at least the keys
                     'train', 'val' that maps to Pytorch data loaders for the
↳ dataset
        criterion: The Loss function
        optimizer: Pytorch optimizer. The algorithm to update weights
        num_epochs: How many epochs to train for
        save_dir: Where to save the best model weights that are found. Using
↳ None will not write anything to disk.

    Returns:
        model: The trained NN
        tr_acc_history: list, training accuracy history. Recording freq: one
↳ epoch.
        val_acc_history: list, validation accuracy history. Recording freq: one
↳ epoch.
    """

    val_acc_history = []
    tr_acc_history = []

    best_model_wts = copy.deepcopy(model.state_dict())
    best_acc = 0.0

    for epoch in range(num_epochs):
        print('Epoch {}/{}'.format(epoch, num_epochs - 1))
        print('-' * 10)

        # Each epoch has a training and validation phase
        for phase in ['train', 'val']:
            if phase == 'train':
                model.train() # Set model to training mode
            else:
                model.eval() # Set model to evaluate mode

            # loss and number of correct prediction for the current batch
            running_loss = 0.0
            running_corrects = 0

            # Iterate over data.
            # TQDM has nice progress bars
```

```

for inputs, labels in tqdm(dataloaders[phase]):
    inputs = inputs.to(device)
    labels = labels.to(device)

#####
# TODO:
# Please read all the inputs carefully!
# For "train" phase:
# (i) Compute the outputs using the model
# Also, use the outputs to calculate the class
# predicted by the model,
# Store the predicted class in 'preds'
# (Think: argmax of outputs across a dimension)
# torch.max() might help!
# (ii) Use criterion to store the loss in 'loss'
# (iii) Update the model parameters
# Notes:
# - Don't forget to zero the gradients before beginning the
# loop!
# - "val" phase is the same as train, but without backprop
# - Compute the outputs (Same as "train", calculate 'preds'
# too),
# - Calculate the loss and store it in 'loss'
#####

if phase == 'train':
    # (i) Compute the outputs using the model
    outputs = model(inputs)
    _, preds = torch.max(outputs, 1)

    # (ii) Use criterion to store the loss in 'loss'
    loss = criterion(outputs, labels)

    # (iii) Update the model parameters
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
else:
    with torch.no_grad():
        outputs = model(inputs)
        _, preds = torch.max(outputs, 1)
        loss = criterion(outputs, labels)

#####
#                                     END OF YOUR CODE
#####

```

```

        # statistics
        running_loss += loss.item() * inputs.size(0)
        running_correcets += torch.sum(preds == labels.data)

    epoch_loss = running_loss / len(dataloaders[phase].dataset)
    epoch_acc = running_correcets.double() / len(dataloaders[phase].
↪dataset)

    print('{} Loss: {:.4f} Acc: {:.4f}'.format(phase, epoch_loss,
↪epoch_acc))

    # deep copy the model
    if phase == 'val' and epoch_acc > best_acc:
        best_acc = epoch_acc
        best_model_wts = copy.deepcopy(model.state_dict())

    # save the best model weights
    # ===== #
    # IMPORTANT:
    # Losing your connection to colab will lead to loss of trained
    # weights.
    # You should download the trained weights to your local machine.
    # Later, you can load these weights directly without needing to
    # train the neural networks again.
    # ===== #
    if save_dir:
        torch.save(best_model_wts, os.path.join(save_dir,
↪model_name + '.pth'))

    # record the train/val accuracies
    if phase == 'val':
        val_acc_history.append(epoch_acc)
    else:
        tr_acc_history.append(epoch_acc)

    print('Best val Acc: {:.4f}'.format(best_acc))

    return model, tr_acc_history, val_acc_history

```

2.5 6.1 (d): Train MiniVGG / MiniVGG-BN

```

[ ]: # Number of classes in the dataset
      # Miniplaces has 100
      num_classes = 100

      # Batch size for training

```

```

batch_size = 128

# Shuffle the input data?
shuffle_datasets = True

# Number of epochs to train for
# During debugging, you can set this parameter to 1
# num_epochs = 1
# Training for 20 epochs. This will take about half an hour.
num_epochs = 20

### IO
# Directory to save weights to
save_dir = "weights"
os.makedirs(save_dir, exist_ok=True)

# get dataloaders and criterion function
input_size = 64
dataloaders = get_dataloaders(input_size, batch_size, shuffle_datasets)
criterion = get_loss()

```

```

[ ]: # Initialize MiniVGG
features = make_layers(cfgs['MiniVGG'], batch_norm=False)
model = VGG(features).to(device)
optimizer = make_optimizer(model)

# Train the model!
vgg, tr_his, val_his = train_model(model=model, dataloaders=dataloaders,
    ↪ criterion=criterion, optimizer=optimizer,
    save_dir=save_dir, num_epochs=num_epochs, model_name='MiniVGG')

```

Epoch 0/19

100%| | 704/704 [01:09<00:00, 10.10it/s]

train Loss: 4.5561 Acc: 0.0137

100%| | 79/79 [00:08<00:00, 9.81it/s]

val Loss: 4.3664 Acc: 0.0303

Epoch 1/19

100%| | 704/704 [01:06<00:00, 10.59it/s]

train Loss: 4.1798 Acc: 0.0517

100%| | 79/79 [00:07<00:00, 10.10it/s]

val Loss: 3.9900 Acc: 0.0836

Epoch 2/19

100%| | 704/704 [01:11<00:00, 9.78it/s]

train Loss: 3.8927 Acc: 0.0906

100%| | 79/79 [00:07<00:00, 10.43it/s]

val Loss: 3.7280 Acc: 0.1183

Epoch 3/19

100%| | 704/704 [01:06<00:00, 10.56it/s]

train Loss: 3.6980 Acc: 0.1224

100%| | 79/79 [00:07<00:00, 10.04it/s]

val Loss: 3.6034 Acc: 0.1441

Epoch 4/19

100%| | 704/704 [01:07<00:00, 10.45it/s]

train Loss: 3.5315 Acc: 0.1489

100%| | 79/79 [00:06<00:00, 12.20it/s]

val Loss: 3.3938 Acc: 0.1747

Epoch 5/19

100%| | 704/704 [01:08<00:00, 10.28it/s]

train Loss: 3.3917 Acc: 0.1715

100%| | 79/79 [00:07<00:00, 10.25it/s]

val Loss: 3.2363 Acc: 0.1951

Epoch 6/19

100%| | 704/704 [01:06<00:00, 10.54it/s]

train Loss: 3.2793 Acc: 0.1918

100%| | 79/79 [00:08<00:00, 9.76it/s]

val Loss: 3.1100 Acc: 0.2173

Epoch 7/19

100%| | 704/704 [01:07<00:00, 10.40it/s]

train Loss: 3.1864 Acc: 0.2104

100%| | 79/79 [00:06<00:00, 12.33it/s]


```

val Loss: 3.0279 Acc: 0.2412
Epoch 8/19
-----
100%|      | 704/704 [01:08<00:00, 10.29it/s]
train Loss: 3.0991 Acc: 0.2256
100%|      | 79/79 [00:07<00:00, 10.45it/s]
val Loss: 3.0106 Acc: 0.2493
Epoch 9/19
-----
100%|      | 704/704 [01:06<00:00, 10.60it/s]
train Loss: 3.0202 Acc: 0.2414
100%|      | 79/79 [00:07<00:00, 9.88it/s]
val Loss: 2.8918 Acc: 0.2653
Epoch 10/19
-----
100%|      | 704/704 [01:08<00:00, 10.25it/s]
train Loss: 2.9394 Acc: 0.2560
100%|      | 79/79 [00:06<00:00, 12.24it/s]
val Loss: 2.8857 Acc: 0.2741
Epoch 11/19
-----
100%|      | 704/704 [01:08<00:00, 10.31it/s]
train Loss: 2.8775 Acc: 0.2691
100%|      | 79/79 [00:07<00:00, 9.99it/s]
val Loss: 2.8116 Acc: 0.2854
Epoch 12/19
-----
100%|      | 704/704 [01:06<00:00, 10.53it/s]
train Loss: 2.8025 Acc: 0.2848
100%|      | 79/79 [00:07<00:00, 10.54it/s]
val Loss: 2.7774 Acc: 0.2971
Epoch 13/19
-----
100%|      | 704/704 [01:08<00:00, 10.30it/s]
train Loss: 2.7401 Acc: 0.2978
100%|      | 79/79 [00:06<00:00, 12.09it/s]

```

```

val Loss: 2.7358 Acc: 0.3077
Epoch 14/19
-----
100%|      | 704/704 [01:08<00:00, 10.21it/s]
train Loss: 2.6844 Acc: 0.3109
100%|      | 79/79 [00:07<00:00,  9.90it/s]
val Loss: 2.7331 Acc: 0.3026
Epoch 15/19
-----
100%|      | 704/704 [01:08<00:00, 10.28it/s]
train Loss: 2.6234 Acc: 0.3198
100%|      | 79/79 [00:06<00:00, 12.04it/s]
val Loss: 2.7383 Acc: 0.3061
Epoch 16/19
-----
100%|      | 704/704 [01:09<00:00, 10.18it/s]
train Loss: 2.5635 Acc: 0.3337
100%|      | 79/79 [00:07<00:00, 10.58it/s]
val Loss: 2.6974 Acc: 0.3139
Epoch 17/19
-----
100%|      | 704/704 [01:07<00:00, 10.38it/s]
train Loss: 2.5034 Acc: 0.3456
100%|      | 79/79 [00:08<00:00,  9.83it/s]
val Loss: 2.6907 Acc: 0.3143
Epoch 18/19
-----
100%|      | 704/704 [01:08<00:00, 10.23it/s]
train Loss: 2.4564 Acc: 0.3563
100%|      | 79/79 [00:06<00:00, 12.26it/s]
val Loss: 2.6770 Acc: 0.3167
Epoch 19/19
-----
100%|      | 704/704 [01:08<00:00, 10.23it/s]
train Loss: 2.3939 Acc: 0.3650
100%|      | 79/79 [00:07<00:00,  9.90it/s]

```

```
val Loss: 2.7336 Acc: 0.3134
Best val Acc: 0.316700
```

```
[ ]: # Initialize MiniVGG-BN
features = make_layers(cfgs['MiniVGG-BN'], batch_norm=True)
model = VGG(features).to(device)
optimizer = make_optimizer(model)

# Train the model!
vgg_BN, tr_his_BN, val_his_BN = train_model(model=model,
    ↪dataloaders=dataloaders, criterion=criterion, optimizer=optimizer,
    save_dir=save_dir, num_epochs=num_epochs, model_name='MiniVGG-BN')
```

Epoch 0/19

```
-----
100%|      | 704/704 [01:10<00:00, 10.03it/s]
train Loss: 4.2788 Acc: 0.0384
100%|      | 79/79 [00:06<00:00, 12.35it/s]
val Loss: 3.9031 Acc: 0.0775
Epoch 1/19
```

```
-----
100%|      | 704/704 [01:09<00:00, 10.18it/s]
train Loss: 3.7450 Acc: 0.1082
100%|      | 79/79 [00:07<00:00, 9.98it/s]
val Loss: 3.6023 Acc: 0.1407
Epoch 2/19
```

```
-----
100%|      | 704/704 [01:08<00:00, 10.34it/s]
train Loss: 3.5002 Acc: 0.1476
100%|      | 79/79 [00:06<00:00, 11.44it/s]
val Loss: 3.2825 Acc: 0.1877
Epoch 3/19
```

```
-----
100%|      | 704/704 [01:08<00:00, 10.26it/s]
train Loss: 3.3541 Acc: 0.1746
100%|      | 79/79 [00:06<00:00, 12.42it/s]
```

```

val Loss: 3.1767 Acc: 0.2071
Epoch 4/19
-----
100%|      | 704/704 [01:08<00:00, 10.24it/s]
train Loss: 3.2457 Acc: 0.1949
100%|      | 79/79 [00:07<00:00, 10.13it/s]
val Loss: 3.0943 Acc: 0.2228
Epoch 5/19
-----
100%|      | 704/704 [01:09<00:00, 10.19it/s]
train Loss: 3.1566 Acc: 0.2115
100%|      | 79/79 [00:06<00:00, 12.46it/s]
val Loss: 3.0436 Acc: 0.2411
Epoch 6/19
-----
100%|      | 704/704 [01:08<00:00, 10.24it/s]
train Loss: 3.0834 Acc: 0.2246
100%|      | 79/79 [00:07<00:00, 10.17it/s]
val Loss: 2.9787 Acc: 0.2493
Epoch 7/19
-----
100%|      | 704/704 [01:06<00:00, 10.54it/s]
train Loss: 3.0222 Acc: 0.2373
100%|      | 79/79 [00:07<00:00, 10.45it/s]
val Loss: 2.9030 Acc: 0.2628
Epoch 8/19
-----
100%|      | 704/704 [01:07<00:00, 10.36it/s]
train Loss: 2.9530 Acc: 0.2528
100%|      | 79/79 [00:06<00:00, 12.10it/s]
val Loss: 2.9327 Acc: 0.2605
Epoch 9/19
-----
100%|      | 704/704 [01:08<00:00, 10.33it/s]
train Loss: 2.8941 Acc: 0.2654
100%|      | 79/79 [00:07<00:00, 9.95it/s]

```

```

val Loss: 2.7988 Acc: 0.2874
Epoch 10/19
-----
100%|      | 704/704 [01:07<00:00, 10.38it/s]
train Loss: 2.8388 Acc: 0.2758
100%|      | 79/79 [00:07<00:00, 10.65it/s]
val Loss: 2.7972 Acc: 0.2873
Epoch 11/19
-----
100%|      | 704/704 [01:08<00:00, 10.32it/s]
train Loss: 2.7812 Acc: 0.2872
100%|      | 79/79 [00:06<00:00, 12.26it/s]
val Loss: 2.6878 Acc: 0.3130
Epoch 12/19
-----
100%|      | 704/704 [01:08<00:00, 10.34it/s]
train Loss: 2.7226 Acc: 0.2993
100%|      | 79/79 [00:07<00:00, 9.98it/s]
val Loss: 2.7751 Acc: 0.3005
Epoch 13/19
-----
100%|      | 704/704 [01:08<00:00, 10.29it/s]
train Loss: 2.6723 Acc: 0.3096
100%|      | 79/79 [00:06<00:00, 11.66it/s]
val Loss: 2.7312 Acc: 0.3096
Epoch 14/19
-----
100%|      | 704/704 [01:08<00:00, 10.21it/s]
train Loss: 2.6211 Acc: 0.3214
100%|      | 79/79 [00:06<00:00, 12.39it/s]
val Loss: 2.6963 Acc: 0.3151
Epoch 15/19
-----
100%|      | 704/704 [01:08<00:00, 10.28it/s]
train Loss: 2.5772 Acc: 0.3302
100%|      | 79/79 [00:07<00:00, 10.00it/s]

```

```

val Loss: 2.6614 Acc: 0.3207
Epoch 16/19
-----
100%|      | 704/704 [01:08<00:00, 10.21it/s]
train Loss: 2.5357 Acc: 0.3377
100%|      | 79/79 [00:06<00:00, 12.09it/s]
val Loss: 2.5899 Acc: 0.3341
Epoch 17/19
-----
100%|      | 704/704 [01:10<00:00, 10.02it/s]
train Loss: 2.4872 Acc: 0.3486
100%|      | 79/79 [00:07<00:00, 9.89it/s]
val Loss: 2.5513 Acc: 0.3413
Epoch 18/19
-----
100%|      | 704/704 [01:09<00:00, 10.14it/s]
train Loss: 2.4446 Acc: 0.3587
100%|      | 79/79 [00:06<00:00, 12.33it/s]
val Loss: 2.5655 Acc: 0.3383
Epoch 19/19
-----
100%|      | 704/704 [01:09<00:00, 10.09it/s]
train Loss: 2.4016 Acc: 0.3640
100%|      | 79/79 [00:08<00:00, 9.83it/s]
val Loss: 2.5361 Acc: 0.3439
Best val Acc: 0.343900

```

```

[ ]: x = np.arange(num_epochs)
      # train/val accuracies for MiniVGG
      plt.figure()
      plt.plot(x, torch.tensor(tr_his, device = 'cpu'))
      plt.plot(x, torch.tensor(val_his, device = 'cpu'))
      plt.legend(['Training top1 accuracy', 'Validation top1 accuracy'])
      plt.xticks(x)
      plt.xlabel('Epoch')
      plt.ylabel('Top1 Accuracy')
      plt.title('MiniVGG')
      plt.show()

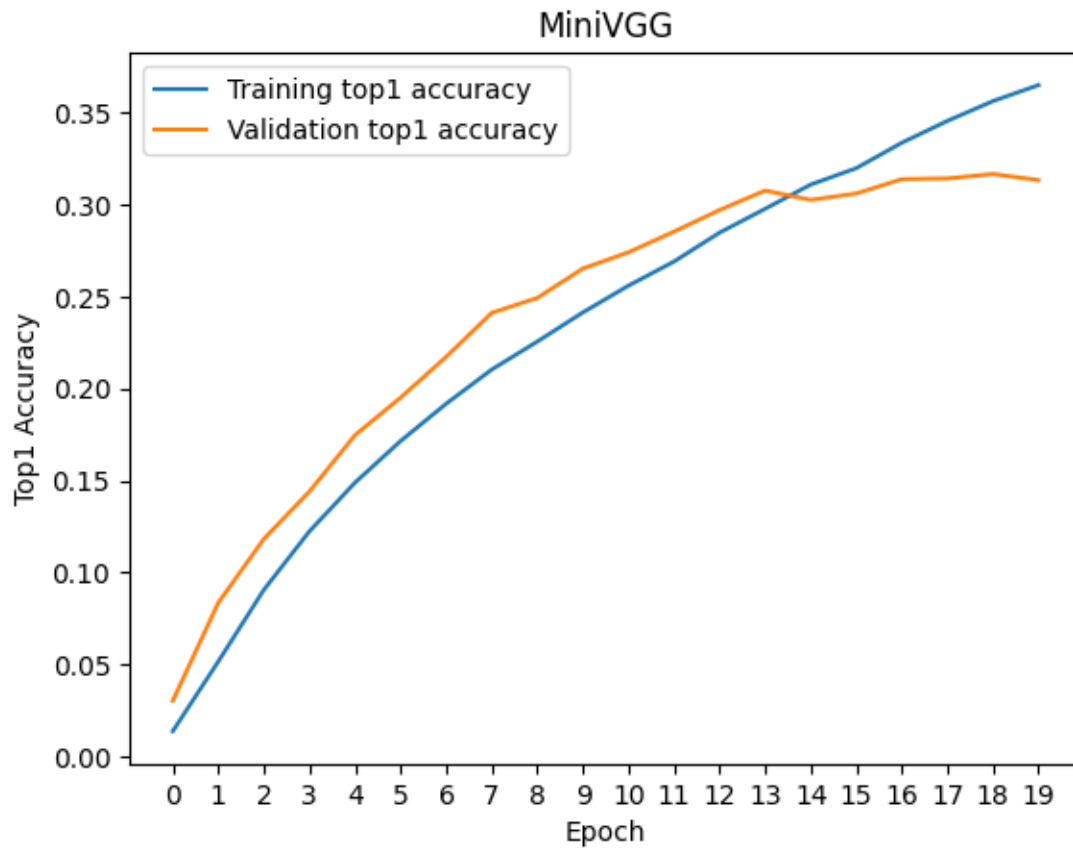
```

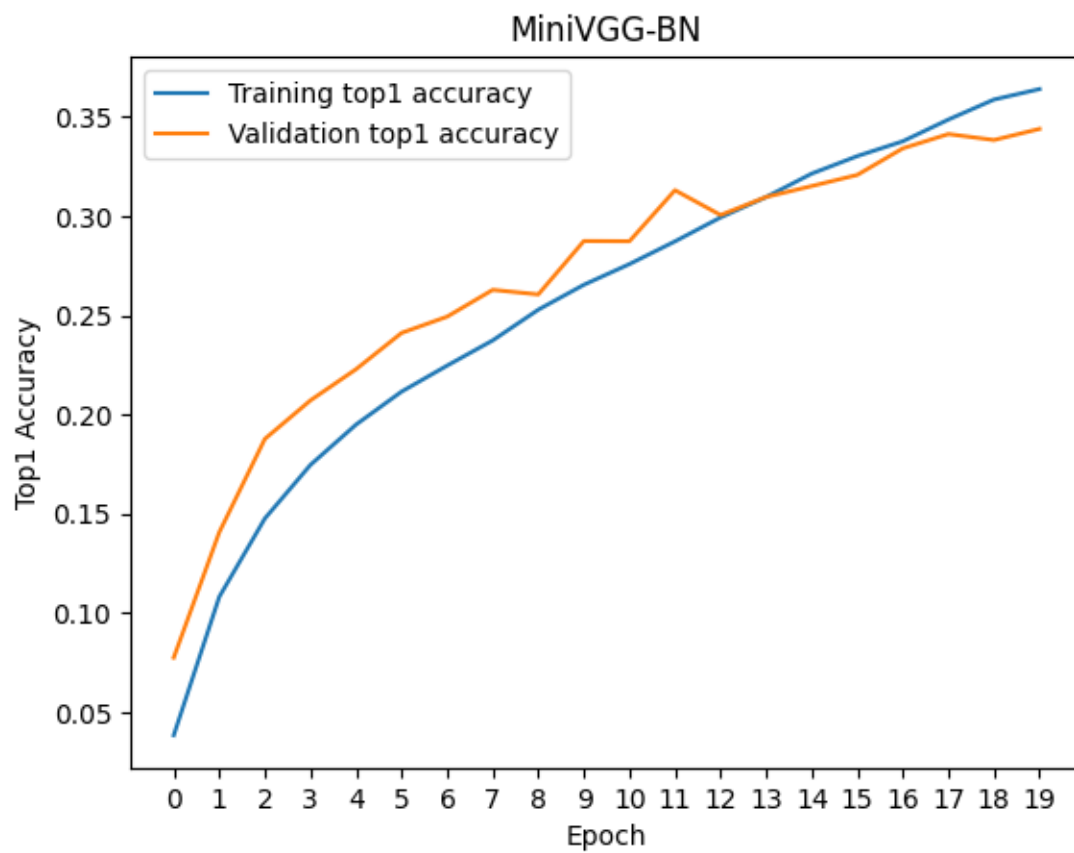
```

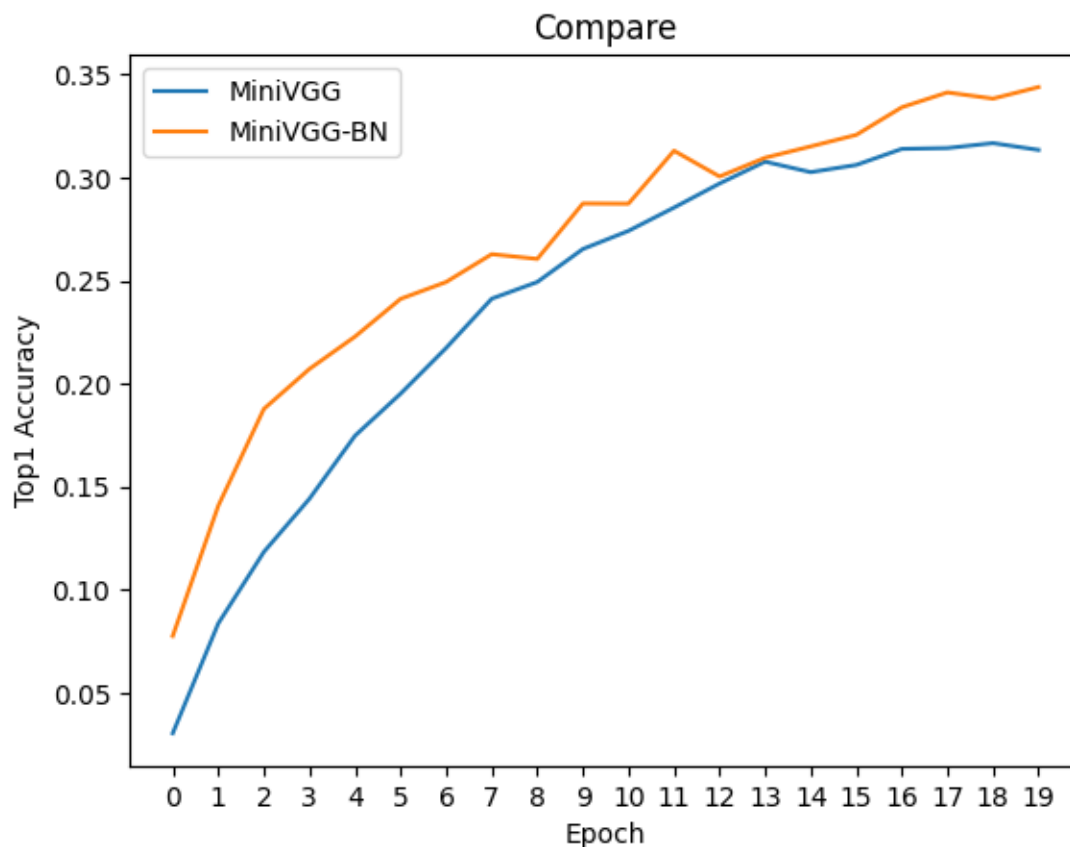
# train/val accuracies for MiniVGG-BN
plt.plot(x, torch.tensor(tr_his_BN, device = 'cpu'))
plt.plot(x, torch.tensor(val_his_BN, device = 'cpu'))
plt.legend(['Training top1 accuracy', 'Validation top1 accuracy'])
plt.xticks(x)
plt.xlabel('Epoch')
plt.ylabel('Top1 Accuracy')
plt.title('MiniVGG-BN')
plt.show()

# compare val accuracies of MiniVGG and MiniVGG-BN
plt.plot(x, torch.tensor(val_his, device = 'cpu'))
plt.plot(x, torch.tensor(val_his_BN, device = 'cpu'))
plt.legend(['MiniVGG', 'MiniVGG-BN'])
plt.xticks(x)
plt.xlabel('Epoch')
plt.ylabel('Top1 Accuracy')
plt.title('Compare')
plt.show()

```







2.5.1 TODO: Summarize the effect of batch normalization:

```
[ ]: # Answer here
Batch normalization makes model train faster and more stable by mitigating
↳vanishing/exploding gradients in deep networks.
```

```
[ ]: pickle.dump(tr_his, open('tr_his.pkl', 'wb'))
pickle.dump(tr_his_BN, open('tr_his_BN.pkl', 'wb'))
pickle.dump(val_his, open('val_his.pkl', 'wb'))
pickle.dump(val_his_BN, open('val_his_BN.pkl', 'wb'))
```

2.6 6.1(e) Measure top1 and top5 accuracies of MiniVGG and MiniVGG-BN

Definition of top-k accuracy: if the correct label is within the *top k* predicted classes according to the network output scores, we count the prediction by the neural network as a correct prediction.

```
[ ]: def accuracy(output, target, topk=(1,)):
    """
    Computes the accuracy over the k top predictions for the specified values
    of k.
```

Args:

output: pytorch tensor, (batch_size x num_classes). Outputs of the network for one batch.

target: pytorch tensor, (batch_size,). True labels for one batch.

Returns:

res: list. Accuracies corresponding to topk[0], topk[1], ...

"""

```
with torch.no_grad():
    maxk = max(topk)
    batch_size = target.size(0)

    _, pred = output.topk(maxk, 1, True, True)
    pred = pred.t()
    correct = pred.eq(target.view(1, -1).expand_as(pred))

    res = []
    for k in topk:
        correct_k = correct[:k].reshape(-1).float().sum(0, keepdim=True)
        res.append(correct_k.mul_(100.0 / batch_size))
    return res
```

```
def test(model, dataloader):
```

```
    model.eval()
```

```
    top1_acc = []
```

```
    top5_acc = []
```

```
    with torch.no_grad():
```

```
        for inputs, labels in dataloader:
```

```
            inputs = inputs.to(device)
```

```
            labels = labels.to(device)
```

```
            outputs = model(inputs)
```

```
            res = accuracy(outputs, labels, topk=(1, 5))
```

```
            top1_acc.append(res[0] * len(outputs))
```

```
            top5_acc.append(res[1] * len(outputs))
```

```
    print('Top-1 accuracy {}%, Top-5 accuracy {}%'.format(sum(top1_acc).item()/
↪10000, sum(top5_acc).item()/10000))
```

```
[ ]: ##### To pass the test, both networks should have Top-5 accuracy above 55% #####
vgg_BN.load_state_dict(torch.load('./weights/MiniVGG-BN.pth'))
```

```
vgg.load_state_dict(torch.load('./weights/MiniVGG.pth'))

test(vgg_BN, dataloaders['test'])
test(vgg, dataloaders['test'])
```

Top-1 accuracy 34.23%, Top-5 accuracy 65.39%
 Top-1 accuracy 32.18%, Top-5 accuracy 62.38%

2.7 6.2 (a-b): Build small ResNet model (Optional)

Please follow this figure to build the Residual Block and the Resnet model. We already implemented the back bone of the resnet model. You have to code the ResNet Block and the classifier part which are shown in the figure.

ResNet architecture

```
[ ]: class ResidualBlock(nn.Module):
    def __init__(self, in_channels, out_channels, stride = 1):
        super(ResidualBlock, self).__init__()

        #####
        # TODO: Code the residual block as depicted in the above figure. You
        # should use      #
        # nn.Sequential().
        #
        #####

        self.residual_block = nn.Sequential(
            nn.Conv2d(in_channels, out_channels, kernel_size=3, stride=stride,
            padding=1),
            nn.BatchNorm2d(out_channels),
            nn.ReLU(),
            nn.Conv2d(out_channels, out_channels, kernel_size=3, stride=stride,
            padding=1),
            nn.BatchNorm2d(out_channels)
        )

        self.identity = nn.Identity()
        self.relu = nn.ReLU(inplace=False)

        #####
        #                               END OF YOUR CODE                               #
        #####

    def forward(self, x):
```

```

    #####
    # TODO: Code the forward pass for the residual block as depicted in the
    # above figure.
    # Note: The relu activation function is after the skip connection.
    #
    #####

    out = self.residual_block(x) + self.identity(x)
    out = self.relu(out)

    #####
    #                                END OF YOUR CODE                                #
    #####
    return out

```

```

[ ]: class ResNet(nn.Module):
    def __init__(self, block, layers, num_classes = 10):
        super(ResNet, self).__init__()

        #####
        # Construct the neural net architecture for the resnet model. You
        # should use nn.Sequential().
        # Note: We already implemented most of the network you just need to
        # code the initial layers and insert the residual blocks.
        #####

        self.backbone = nn.Sequential(

            #####
            # TODO: Code the initial layers i.e the the strided convolution
            # layer, batchnorm, relu, maxpool layer and the residual blocks
            # Hint: you have to make use of the "block" variable.
            #####

            nn.Conv2d(3, 64, kernel_size = 7, stride = 2, padding = 3,
            bias=False),
            nn.BatchNorm2d(64),
            nn.ReLU(inplace=False),
            nn.MaxPool2d(kernel_size = 2, stride = 2, padding=1),

            *[block(64, 64) for _ in range(layers[0])],

```

```

#####
→ #####
#                                END OF YOUR CODE
→ #

#####
nn.Sequential(
    nn.Conv2d(64, 128, kernel_size = 3, stride = 2, padding = 1),
    nn.BatchNorm2d(128),
    nn.ReLU()),

#####
→ #####
# TODO: Insert the residual block please follow the figure for the
→ number of channels and the flow of layers.

#####
→ #####

    *[block(128, 128) for _ in range(layers[1])],

#####
→ #####
#                                END OF YOUR CODE
→ #

#####
nn.Sequential(
    nn.Conv2d(128, 256, kernel_size = 3, stride = 2, padding = 1),
    nn.BatchNorm2d(256),
    nn.ReLU()),

#####
→ #####
# TODO: Insert the residual block please follow the figure for the
→ number of channels and the flow of layers.

#####
→ #####

    *[block(256, 256) for _ in range(layers[2])],

#####
→ #####
#                                END OF YOUR CODE
→ #

#####
nn.Sequential(

```

```

        nn.Conv2d(256, 512, kernel_size = 3, stride = 2, padding = 1),
        nn.BatchNorm2d(512),
        nn.ReLU()),
    )
    #####
    # TODO: Insert the residual block please follow the figure for the
    # number of channels and the flow of layers.
    #####
    *[block(512, 512) for _ in range(layers[3])],
    )
    #####
    #                                     END OF YOUR CODE
    #
    #####

    )
    self.avgpool = nn.AvgPool2d(2, stride=1)
    self.fc = nn.Linear(512, num_classes)

    def forward(self, x):
        x = self.backbone(x)
        x = self.avgpool(x)
        x = x.view(x.size(0), -1)
        x = self.fc(x)

    return x

```

```

[ ]: resnet = ResNet(ResidualBlock, [1, 1, 1, 1], 100)
print(resnet)
print('Number of trainable parameters {}'.format(count_params(resnet)))

```

```

ResNet(
  (backbone): Sequential(
    (0): Conv2d(3, 64, kernel_size=(7, 7), stride=(2, 2), padding=(3, 3),
    bias=False)
    (1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True,
    track_running_stats=True)
    (2): ReLU()
    (3): MaxPool2d(kernel_size=3, stride=2, padding=1, dilation=1,
    ceil_mode=False)
    (4): ResidualBlock(
      (residual_block): Sequential(

```

```

        (0): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
        (1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
        (2): ReLU()
        (3): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
        (4): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    )
    (identity): Identity()
    (relu): ReLU()
)
(5): Sequential(
  (0): Conv2d(64, 128, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1))
  (1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
  (2): ReLU()
)
(6): ResidualBlock(
  (residual_block): Sequential(
    (0): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (2): ReLU()
    (3): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (4): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
  )
  (identity): Identity()
  (relu): ReLU()
)
(7): Sequential(
  (0): Conv2d(128, 256, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1))
  (1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
  (2): ReLU()
)
(8): ResidualBlock(
  (residual_block): Sequential(
    (0): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (2): ReLU()
    (3): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (4): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
  )
  (identity): Identity()
  (relu): ReLU()
)

```

```

    )
    (9): Sequential(
      (0): Conv2d(256, 512, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1))
      (1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (2): ReLU()
    )
    (10): ResidualBlock(
      (residual_block): Sequential(
        (0): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
        (1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
        (2): ReLU()
        (3): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
        (4): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      )
      (identity): Identity()
      (relu): ReLU()
    )
  )
  (avgpool): AvgPool2d(kernel_size=2, stride=1, padding=0)
  (fc): Linear(in_features=512, out_features=100, bias=True)
)
Number of trainable parameters 7884452

```

2.8 6.2 (c): Train ResNet (Optional)

```

[ ]: # Initialize ResNet
resnet = ResNet(ResidualBlock, [1, 1, 1, 1], num_classes).to(device)
optimizer = make_optimizer(resnet)

# Train the model!
resnet, tr_his_res, val_his_res = train_model(model=resnet,
↪ dataloaders=dataloaders, criterion=criterion, optimizer=optimizer,
    save_dir=save_dir, num_epochs=num_epochs, model_name='ResNet')

```

Epoch 0/19

100%| | 704/704 [01:08<00:00, 10.31it/s]

train Loss: 3.6702 Acc: 0.1347

100%| | 79/79 [00:06<00:00, 11.92it/s]

val Loss: 3.4361 Acc: 0.1737

Epoch 1/19

100%| | 704/704 [01:08<00:00, 10.30it/s]

train Loss: 3.1385 Acc: 0.2228

100%| | 79/79 [00:07<00:00, 10.61it/s]

val Loss: 3.2496 Acc: 0.2075

Epoch 2/19

100%| | 704/704 [01:07<00:00, 10.37it/s]

train Loss: 2.8645 Acc: 0.2774

100%| | 79/79 [00:08<00:00, 9.75it/s]

val Loss: 3.0428 Acc: 0.2522

Epoch 3/19

100%| | 704/704 [01:08<00:00, 10.27it/s]

train Loss: 2.6317 Acc: 0.3227

100%| | 79/79 [00:06<00:00, 12.34it/s]

val Loss: 2.9483 Acc: 0.2670

Epoch 4/19

100%| | 704/704 [01:08<00:00, 10.21it/s]

train Loss: 2.4247 Acc: 0.3639

100%| | 79/79 [00:07<00:00, 9.98it/s]

val Loss: 2.9473 Acc: 0.2784

Epoch 5/19

100%| | 704/704 [01:07<00:00, 10.45it/s]

train Loss: 2.2017 Acc: 0.4123

100%| | 79/79 [00:07<00:00, 10.47it/s]

val Loss: 2.8783 Acc: 0.2942

Epoch 6/19

100%| | 704/704 [01:08<00:00, 10.21it/s]

train Loss: 1.9644 Acc: 0.4640

100%| | 79/79 [00:06<00:00, 12.13it/s]

val Loss: 2.9725 Acc: 0.2869

Epoch 7/19

100%| | 704/704 [01:09<00:00, 10.19it/s]

train Loss: 1.6944 Acc: 0.5265

100%| | 79/79 [00:08<00:00, 9.81it/s]

val Loss: 3.1236 Acc: 0.2794

Epoch 8/19

100%| | 704/704 [01:08<00:00, 10.32it/s]

train Loss: 1.3935 Acc: 0.6006

100%| | 79/79 [00:07<00:00, 11.25it/s]

val Loss: 3.2850 Acc: 0.2793

Epoch 9/19

100%| | 704/704 [01:08<00:00, 10.23it/s]

train Loss: 1.0834 Acc: 0.6841

100%| | 79/79 [00:06<00:00, 11.99it/s]

val Loss: 3.5313 Acc: 0.2797

Epoch 10/19

100%| | 704/704 [01:09<00:00, 10.10it/s]

train Loss: 0.7642 Acc: 0.7730

100%| | 79/79 [00:07<00:00, 9.91it/s]

val Loss: 4.0037 Acc: 0.2613

Epoch 11/19

100%| | 704/704 [01:07<00:00, 10.49it/s]

train Loss: 0.5076 Acc: 0.8482

100%| | 79/79 [00:07<00:00, 10.61it/s]

val Loss: 4.1093 Acc: 0.2736

Epoch 12/19

100%| | 704/704 [01:08<00:00, 10.31it/s]

train Loss: 0.3045 Acc: 0.9113

100%| | 79/79 [00:06<00:00, 12.36it/s]

val Loss: 4.4269 Acc: 0.2715

Epoch 13/19

100%| | 704/704 [01:09<00:00, 10.19it/s]

train Loss: 0.2176 Acc: 0.9392

100%| | 79/79 [00:07<00:00, 9.91it/s]

val Loss: 4.5842 Acc: 0.2709

Epoch 14/19

100%| | 704/704 [01:08<00:00, 10.32it/s]

train Loss: 0.0988 Acc: 0.9762

100%| | 79/79 [00:07<00:00, 10.53it/s]

val Loss: 4.7034 Acc: 0.2761

Epoch 15/19

100%| | 704/704 [01:07<00:00, 10.36it/s]

train Loss: 0.0711 Acc: 0.9844

100%| | 79/79 [00:06<00:00, 12.32it/s]

val Loss: 4.7627 Acc: 0.2853

Epoch 16/19

100%| | 704/704 [01:08<00:00, 10.21it/s]

train Loss: 0.0375 Acc: 0.9937

100%| | 79/79 [00:07<00:00, 9.90it/s]

val Loss: 4.8449 Acc: 0.2841

Epoch 17/19

100%| | 704/704 [01:06<00:00, 10.60it/s]

train Loss: 0.0135 Acc: 0.9988

100%| | 79/79 [00:08<00:00, 9.87it/s]

val Loss: 4.8368 Acc: 0.2934

Epoch 18/19

100%| | 704/704 [01:08<00:00, 10.30it/s]

train Loss: 0.0080 Acc: 0.9993

100%| | 79/79 [00:06<00:00, 12.33it/s]

val Loss: 4.8747 Acc: 0.2896

Epoch 19/19

100%| | 704/704 [01:07<00:00, 10.36it/s]

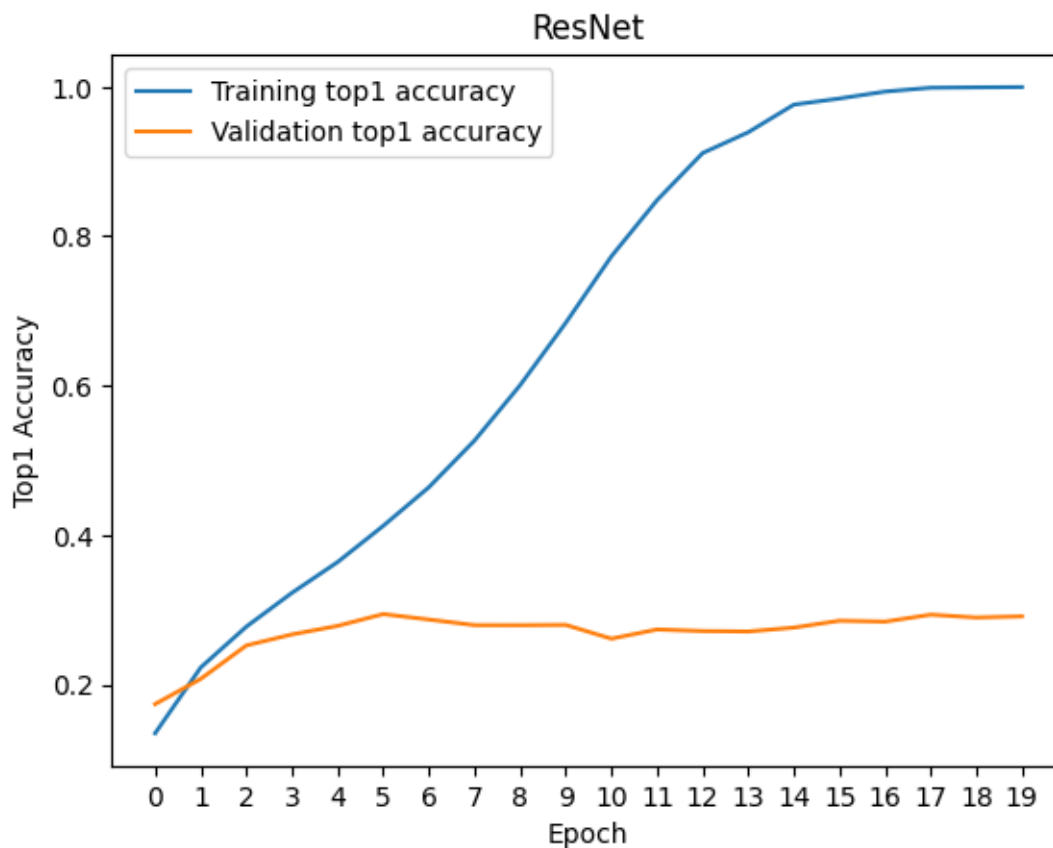
train Loss: 0.0043 Acc: 0.9997

100%| | 79/79 [00:07<00:00, 11.08it/s]

val Loss: 4.9146 Acc: 0.2912

Best val Acc: 0.294200

```
[ ]: # train/val accuracies for ResNet
plt.figure()
plt.plot(x, torch.tensor(tr_his_res, device = 'cpu'))
plt.plot(x, torch.tensor(val_his_res, device = 'cpu'))
plt.legend(['Training top1 accuracy', 'Validation top1 accuracy'])
plt.xticks(x)
plt.xlabel('Epoch')
plt.ylabel('Top1 Accuracy')
plt.title('ResNet')
plt.show()
```



```
[ ]: pickle.dump(tr_his_res, open('tr_his_res.pkl', 'wb'))
pickle.dump(val_his_res, open('val_his_res.pkl', 'wb'))
```

```
[ ]: ##### (Optional) Network should have Top-5 accuracy above 55% #####
resnet.load_state_dict(torch.load('./weights/ResNet.pth'))
test(resnet, dataloaders['test'])
```

Top-1 accuracy 29.96%, Top-5 accuracy 59.81%

3 Convert Notebook to PDF

```
[1]: # generate pdf
# Please provide the full path of the notebook file below
# Important: make sure that your file name does not contain spaces!
import os
notebookpath = '/content/drive/My Drive/EECS 442 - Computer Vision/Hw/hw6/
↳ pohsun_40183452.ipynb' # Ex: notebookpath = '/content/drive/My Drive/Colab_
↳ Notebooks/EECS 442 Fall 2025 - PS6.ipynb'
drive_mount_point = '/content/drive/'
from google.colab import drive
drive.mount(drive_mount_point)
file_name = notebookpath.split('/')[-1]
get_ipython().system("apt update && apt install texlive-xetex_
↳ texlive-fonts-recommended texlive-generic-recommended")
get_ipython().system("pip install py pandoc")
get_ipython().system("apt-get install texlive texlive-xetex texlive-latex-extra_
↳ pandoc")
get_ipython().system("jupyter nbconvert --to PDF {}".format(notebookpath.
↳ replace(' ', '\\ ')))
from google.colab import files
files.download(notebookpath.split('.')[0]+' .pdf')
```

```
Mounted at /content/drive/
Hit:1 https://cli.github.com/packages stable InRelease
Get:2 http://security.ubuntu.com/ubuntu jammy-security InRelease [129 kB]
Get:3 https://cloud.r-project.org/bin/linux/ubuntu jammy-cran40/ InRelease
[3,632 B]
Get:4 https://developer.download.nvidia.com/compute/cuda/repos/ubuntu2204/x86_64
InRelease [1,581 B]
Get:5 https://r2u.stat.illinois.edu/ubuntu jammy InRelease [6,555 B]
Hit:6 http://archive.ubuntu.com/ubuntu jammy InRelease
Get:7 http://archive.ubuntu.com/ubuntu jammy-updates InRelease [128 kB]
Get:8 https://developer.download.nvidia.com/compute/cuda/repos/ubuntu2204/x86_64
Packages [2,085 kB]
Hit:9 https://ppa.launchpadcontent.net/deadsnakes/ppa/ubuntu jammy InRelease
Hit:10 https://ppa.launchpadcontent.net/graphics-drivers/ppa/ubuntu jammy
InRelease
Hit:11 https://ppa.launchpadcontent.net/ubuntugis/ppa/ubuntu jammy InRelease
Get:12 http://security.ubuntu.com/ubuntu jammy-security/main amd64 Packages
[3,468 kB]
```